

Reviewer Pack — Apolar Span of Clause-Product Polynomial Bounds DPLL Leaves

Entry fe0af6ae6d43 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-27 15:25:05 UTC

Apolar Span of Clause-Product Polynomial Bounds DPLL Leaves

Entry ID: fe0af6ae6d43 **Verdict:** INCONCLUSIVE **Recorded:** 2026-04-27 15:25:05 UTC

1. Conjecture

Field A (mathematical branch): Apolarity / partial-derivative-span (Macaulay inverse system, catalecticant rank) — the Iarrobino-Kanev / Landsberg-Ottaviani lineage that underlies GCT border-rank obstructions for permanent vs. determinant. **Field B** (complexity object): Lexicographic DPLL search-tree leaf count on small unsatisfiable 3-CNF formulas.

Statement:

For an unsatisfiable 3-CNF F on $n \leq 10$ variables with m clauses, attach to each clause C the affine linear form $L_C := \text{off}(C) + \sum_{\text{positive } x_i \in C} x_i - \sum_{\text{negative } \neg x_i \in C} x_i$, where $\text{off}(C) = (\#\text{negative literals in } C)$; thus $L_C(x)$ equals the number of true literals of C at $x \in \{0,1\}^n$. Let $f_F := \prod_{C \in F} L_C \in Q[x_1, \dots, x_n]$, let $\pi : Q[x] \rightarrow Q[x]/(x_i^2 - x_i) \cong Q^{\binom{n}{2}}$ be the squarefree-reduction map, and define $\mu(F) := \dim_Q \text{span}\{\pi(\partial\{x_i\} f_F) : i=1, \dots, n\} \subseteq Q^{\binom{n}{2}}$, computed via the identity $\pi(\partial\{x_i\} f_F) = \sum \{C\} s(C,i) \cdot \pi(\prod_{C' \neq C} L_{C'})$ where $s(C,i) \in \{-1, 0, +1\}$ is the sign of variable x_i in clause C . Conjecture: for every unsatisfiable F in this regime, $\mu(F) \geq \lceil \log_2 D_F \rceil$, where D_F is the leaf count of depth-first DPLL with unit propagation that always branches on the lowest-indexed unassigned variable (positive branch first); a single unsatisfiable F with $\mu(F) < \lceil \log_2 D_F \rceil$ refutes it.

Rationale (proposer's reasoning):

Apolarity converts lower bounds on partial-derivative-span dimension into border-rank obstructions, the engine of Landsberg-Ottaviani's permanent lower bounds inside GCT. Each clause is genuinely a linear form, so f_F is exactly the kind of degree- m product whose derivative-span GCT analyzes for orbit-closure obstructions; the squarefree projection π is the natural finite Macaulay quotient. Linking $\mu(F)$ to DPLL leaves tests whether the algebraic rigidity that obstructs orbit-closure containment is the same rigidity that obstructs DPLL collapse — a candidate computational shadow of GCT obstructions.

Taxonomy category: GCT (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 96152a1d2f89e120

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across ~30 random UNSAT 3-CNFs (mix of $n=8, m=42$ and $n=10, m=50$), each certified UNSAT, compute $\mu(F)$ and $\lceil \log_2 D_F \rceil$. Conjecture is SUPPORTED if $\geq 90\%$ satisfy $\mu(F) \geq \lceil \log_2 D_F \rceil$ and ≥ 1 instance has $D_F \geq 8$; FALSIFIED if any single instance yields $\mu(F) < \lceil \log_2 D_F \rceil$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.92	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.86	SAFE	SAFE
ALGEBRIZATION	SAFE	0.78	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - apolarity catalecticant DPLL resolution lower bound CNF - partial derivative span polynomial unsatisfiable 3-CNF proof complexity - Macaulay inverse system clause product polynomial DPLL tree size

Top relevant hits considered: - [<http://arxiv.org/abs/0906.0693v3>] An improved lower bound on the counterfeit coins problem - [<http://arxiv.org/abs/1007.1875v2>] Lower Bounds for Quantum Oblivious Transfer - [<http://arxiv.org/abs/1801.08709v2>] Adaptive Lower Bound for Testing Monotonicity on the Line - [<http://arxiv.org/abs/2405.16149v3>] Small unsatisfiable k -CNFs with bounded literal occurrence - [<http://arxiv.org/abs/1203.3706v2>] On the Complexity of Computing Minimal Unsatisfiable LTL formulas - [<http://arxiv.org/abs/2412.05017v5>] Reduction from the partition problem: Dynamic lot sizing problem with polynomial complexity - [<http://arxiv.org/abs/math/0007142v2>] An excursion from enumerative geometry to solving systems of polynomial equations with Macaulay 2 - [<http://arxiv.org/abs/1906.07508v1>] Subsumption-driven clause learning with DPLL+restarts

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, $\leq 90s$ wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
import sys
import json

def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

def lcm(a, b):
```

```

    return abs(a * b) // gcd(a, b)

def matrix_multiply(A, B):
    n = len(A)
    C = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            for k in range(n):
                C[i][j] += A[i][k] * B[k][j]
    return C

def gaussian_elimination(M, b):
    n = len(M)
    M_b = [row + [b[i]] for i, row in enumerate(M)]
    for i in range(n):
        max_row = i
        for j in range(i+1, n):
            if abs(M[j][i]) > abs(M[max_row][i]):
                max_row = j
        M_b[i], M_b[max_row] = M_b[max_row], M_b[i]
        pivot = M_b[i][i]
        for j in range(n + 1):
            M_b[i][j] /= pivot
        for j in range(n):
            if i != j:
                factor = M_b[j][i]
                for k in range(n + 1):
                    M_b[j][k] -= factor * M_b[i][k]
    return [row[-1] for row in M_b]

def squarefree_reduction(poly, n):
    result = {}
    for term, coeff in poly.items():
        mask = 0
        for var, sign in enumerate(term):
            if sign > 0:
                mask |= (1 << var)
            else:
                mask &= ~(1 << var)
        result[mask] += coeff
    return result

def derivative(poly, var, n):
    result = {}
    for term, coeff in poly.items():
        new_term = []
        for i in range(n):
            if i != var:
                new_term.append((term[i], term[i]))
        result[tuple(new_term)] += coeff * (1 - 2 * (var + 1 in term))
    return result

def dpll(F, assignment, n):
    if not F:
        return True
    for clause in F:

```

```

    if all(lit in assignment and assignment[lit] == sign for lit, sign in clause):
        continue
    for var, sign in clause:
        new_assignment = assignment.copy()
        new_assignment[var + 1 if sign > 0 else -(var + 1)] = sign
        if dpll(F - {tuple(sorted([var + 1 if sign > 0 else -(var + 1) for sign, var in clause))}):
            return True
    return False

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_3cnf(n, m):
        variables = list(range(1, n + 1))
        clauses = []
        for _ in range(m):
            clause = []
            for _ in range(3):
                var = random.choice(variables)
                sign = random.choice([1, -1])
                if (var, sign) not in clause and (-var, -sign) not in clause:
                    clause.append((var, sign))
            clauses.append(tuple(sorted(clause)))
        return set(clauses)

    def is_unsat(F, n):
        assignment = {}
        return not dpll(F, assignment, n)

    n_values = [8, 10]
    m_values = [42, 50]

    for n in n_values:
        for m in m_values:
            F = generate_3cnf(n, m)
            if is_unsat(F, n):
                D_F = dpll(F, {}, n)
                if D_F < 8:
                    continue
                f_F = {}
                for clause in F:
                    L_C = {tuple(sorted([var + 1 if sign > 0 else -(var + 1) for sign, var in clause])}
                    for C_prime in F - {clause}:
                        L_C = squarefree_reduction(derivative(L_C, var, n), n)
                 $\pi_F$  = {}
                for term, coeff in f_F.items():
                    mask = 0
                    for var, sign in enumerate(term):
                        if sign > 0:
                            mask |= (1 << var)
                        else:
                            mask &= ~(1 << var)
                     $\pi_F$ [mask] += coeff
                 $\mu_F$  = []
                for i in range(n):
                     $\psi_i$  = sum(s(C, i) *  $\pi_F$ [tuple(sorted([var + 1 if sign > 0 else -(var + 1) for

```

```

         $\mu_F$ .append( $\psi_i$ )
    rank = len(set(gaussian_elimination([[term[i] for term in  $\mu_F$  for i in range(n)],
    if rank < math.ceil(math.log2(D_F)):
        return {
            "metric_name": " $\mu(F)$ ",
            "metric_value": rank,
            "instances_tested": 1,
            "conjecture_holds": False,
            "counterexample": f"n={n}, m={m}, D_F={D_F}"
        }
    return {
        "metric_name": " $\mu(F)$ ",
        "metric_value": 0,
        "instances_tested": 0,
        "conjecture_holds": True,
        "counterexample": ""
    }

if __name__ == "__main__":
    seeds = [int(s) for s in sys.argv[1:]] or [11, 23, 37, 53, 71]
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {json.dumps(result)}")
        results.append(result)

    total_metric_value = sum(r["metric_value"] for r in results if "metric_value" in r)
    support_fraction = sum(1 for r in results if r["conjecture_holds"])

    if all(r["conjecture_holds"] for r in results) or support_fraction / len(results) >= 0.9:
        print(f"RESULT: SUPPORTED mean={total_metric_value/len(results):.2f} std=0.00 support_fra")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["con"]
        print(f"RESULT: FALSIFIED counterexample=\"{result['counterexample']}\\" first_failing_seed")
    else:
        print("RESULT: INCONCLUSIVE insufficient data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

ome/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_38d58c5e.py", line 159, in <module>
    result = run_trial(seed)
    ~~~~~
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_38d58c5e.py", line 115, in run_trial
    if is_unsat(F, n):
    ~~~~~
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_38d58c5e.py", line 107, in is_unsat
    return not dpll(F, assignment, n)
    ~~~~~
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_38d58c5e.py", line 85, in dpll
    if dpll(F - {tuple(sorted([var + 1 if sign > 0 else -(var + 1) for sign, var in clause])) for clause in
    ~~~~~

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_38d58c5e.py", line 85, in dpll
    if dpll(F - {tuple(sorted([var + 1 if sign > 0 else -(var + 1) for sign, var in clause])) for clause i
    ~~~~~
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_38d58c5e.py", line 85, in dpll
    if dpll(F - {tuple(sorted([var + 1 if sign > 0 else -(var + 1) for sign, var in clause])) for clause i
    ~~~~~
[Previous line repeated 993 more times]
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_38d58c5e.py", line 80, in dpll
    if all(lit in assignment and assignment[lit] == sign for lit, sign in clause):
    ~~~~~
RecursionError: maximum recursion depth exceeded

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with a RecursionError in the DPLL implementation before producing any data, so no instances were evaluated against the pre-registered criterion. | next: Fix the DPLL routine (iterative implementation or raise recursion limit, and correct the buggy clause-filtering expression) and rerun the 30-instance protocol.

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	249058
2	preregistration	claude_max	opus	0	0	9368
3	novelty	claude_max	opus	0	0	4751
4	novelty	claude_max	opus	0	0	7898
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18830
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17208
7	judge	claude_max	opus	0	0	6980

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 314093 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 2}

(full prompt+response transcripts available in `research/audit/fe0af6ae6d43.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/fe0af6ae6d43.jsonl` for verification of provenance

4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/fe0af6ae6d43.tar.gz` (if generated)